

Shape Truthful Clustering (STC)

Algorithm Definitions

Prof. Dr. Asis Hallab

August 19, 2026

In this document we find the pseudocode definitions of the functions for the Shape Truthful Clustering (STC) variant tangent space.

1 Data Definitions

- Input multivariate vectors as an 2D Fortran real array.
- Logical mask defining the seeds.

2 Numerical Linear Algebra: SVD vs. Eigendecomposition

This module never forms a covariance or Gram matrix explicitly and eigendecomposes it. Forming $C = YY^\top$ (or $M^\top M$ for a comparison between two bases) squares the condition number of the underlying data relative to computing singular values of Y (or M) directly, which disproportionately degrades precision in the *small* eigenvalues – exactly the normal-space ones that `normal_error` and `tangent_scales` depend on. This is not a question of which eigensolver to use: even LAPACK’s most robust symmetric eigensolver cannot recover precision already lost by forming the squared matrix in the first place. The fix is to never form it: compute singular values and vectors of the actual centered data (or comparison) matrix directly.

Two distinct problem shapes occur in this module, with two different recommended routines:

- **Full spectrum** (all D singular values/vectors of a $D \times k$ centered ensemble matrix $Y_{\mathcal{E}}$, needed by `observable`, `normal_error`, and `tangent_scales`): use `dgesdd` (divide-and-conquer SVD), economy mode (`JOBZ='S'`). It is the fastest LAPACK SVD routine for computing full singular vectors, and is standard/portable across any LAPACK/OpenBLAS build. It requires a genuine workspace *query* (call once with `LWORK=-1`, read the optimal size back from `WORK(1)`) rather than a closed-form minimum, plus an integer workspace array (`IWORK`, size $8 \cdot \min(D, k)$) – size both in the `_alloc` layer.
- **Small $d \times d$ matrix** (comparing two tangent bases, e.g. $M = U_{\mathcal{E}_r}^\top U_{\mathcal{E}_{t+1}}$, for principal angles in `accept_ensemble`): use `dgesvd`, not `dgesdd`. At d ’s typical size (the intrinsic dimension, usually 1-5), divide-and-conquer’s speed advantage does not materialize and its extra `IWORK` bookkeeping is pure overhead for no benefit; `dgesvd`’s simpler single-workspace-array bookkeeping wins. Its singular values are $\cos \theta_j$ directly – no squaring, no `sqrt` needed back out. `accept_ensemble` now performs up to $o + 1$ such comparisons per growth step, one per retained reference basis (see “SKG `accept_ensemble`” below), not just one – still all small $d \times d$ problems, so the routine choice is unaffected, only the count.

Do not use `dsyev`, `dsyevd`, or `dsyevr` anywhere in this module. They solve “eigendecompose a given symmetric matrix as accurately as possible,” which is not the problem this module has – it can choose not to form that symmetric matrix at all.

3 Functions / Subroutines

3.1 Coding / Naming convention

Follow `misc/Fortran_Coding_Guides.pdf` (the F42 standard) for general Scientific Kernel (SK) philosophy (section 10.2: purity, no I/O, no `allocate/deallocate` in the numerical core, determinism), and `codegen_guide.md` for the naming scheme and how each procedure is written – it supersedes F42 section 10.3.2’s older `_alloc -> unsuffixed -> _helper` suffix triad, and (as of the 131-`codegen-new` generator) also supersedes this document’s own earlier `_kernel/src/kernel/` naming, which is what `codegen_guide.md` itself once called an implementation. Concretely:

- an implementation is a **pure** procedure named `<name>_impl`, in a module named `tox_<family>_impl` under `src/tox/`: no validation, no `ierr` (except a genuine runtime failure no input check could foresee, e.g. an SVD that fails to converge – `codegen_guide.md` section 5.14), no allocation anywhere in the module;
- every argument’s constraints are stated as `!!/DM_*` documentation (`DM_MIN`, `DM_MAX`, `DM_SENTINEL`, `DM_DEFAULT`, `DM_OUTPUT_FROM`, `DM_PROLOGUE`, ...) – that documentation is the generator’s only input, and the *sole* place validation for that argument is stated;
- the generator derives the plain entry point (validates *and* allocates, published under the implementation’s own unsuffixed name), the expert tier (validates only, caller supplies every work array – published as `_expert`), and the C/Python/R bindings from the implementation and its annotations – none of those are hand-written. Every stored **logical** (an array or a scalar that isn’t a `logical`, `parameter` or a `logical` function result) is declared `logical(c_bool)`, the one-byte kind, not plain `logical`.

Related routines following this pattern are called SK-groups (SKGs). See “Implementation Modules” below for where each SKG’s implementation lives.

3.2 Implementation Modules

Each major step gets its own implementation module under `src/tox/shape_truthful_clustering/`, following `codegen_guide.md` section 5.15 (“a family too big for one file”):

Module file	SKGs
<code>tox_shape_truthful_clustering_impl.F90</code> (parent)	<code>ensemble_identification</code> (its own implementation(s), directly in the parent – see note below)
<code>tox_shape_truthful_clustering_seeding_impl.F90</code>	<code>density_labels</code> , <code>seeds</code>
<code>tox_shape_truthful_clustering_ensemble_growing_impl.F90</code>	<code>ensemble_growth_radius</code> , <code>grow_ensemble</code>
<code>tox_shape_truthful_clustering_observable_impl.F90</code>	<code>observable</code> , <code>normal_error</code> , <code>tangent_scales</code> , <code>ensemble_final_observable</code>
<code>tox_shape_truthful_clustering_accept_impl.F90</code>	<code>accept_ensemble</code>
<code>tox_shape_truthful_clustering_filter_impl.F90</code>	<code>filter_ensembles_by_stop_condition</code> , <code>filter_ensembles_by_dimension</code> , <code>filter_ensembles_by_var_explained</code> , <code>filter_ensembles</code>
<code>tox_shape_truthful_clustering_reconciliation_impl.F90</code>	<code>ensemble_reconciliation</code> (a thin two-call orchestrator: <code>filter_ensembles</code> then this module’s own <code>merge_to_super_ensembles</code>)
<code>tox_shape_truthful_clustering_parameter_estimation_impl.F90</code>	<code>sample_estimator_anchors</code> , <code>grow_estimator_anchor_clouds</code> , <code>estimate_stc_parameters</code>

The parent module holds `ensemble_identification`’s own implementation(s) directly, in addition to use-ing its siblings – a deliberate deviation from section 5.15’s own `tox_data_integration_impl` example, whose parent holds no procedures of its own. `ensemble_identification` is this family’s natural top-level entry point (it orchestrates every other SKG), unlike `data_integration`’s parent, which really is just a bag of siblings with no natural “top” member.

`ensemble_final_observable` lives in the *observable* module, not the reconciliation family it was introduced to serve, and not the parent module either, even though it operates on `ensemble_identification`’s own history

output: both `tox_stc_json` (report-layer “final ensemble state” extraction) and `filter_ensembles` (see “Ensemble Reconciliation” below) need the exact same computation, and the parent module already *uses* the reconciliation module – which now, in turn, *uses* the filter module – so placing this kernel anywhere reachable *from* the parent (including the parent itself) would close a cycle. The observable module has no dependency on any of its siblings, making it the one safe home both consumers can *use* without one.

3.3 Seeding

Seeding requires two dedicated SKGs: one to assign every vector a local density label, the other to greedily select seeds from those labels.

3.3.1 SKG `density_labels`

Receives an optional k_{density} (default 30 – the same default `calc_ensemble_growth_radius`’s own k_{min} uses, see “Local Radius Identification” below, but a genuinely independent argument: `density_labels` and `seeds` are called before `ensemble_identification` in the pipeline, so there is no single call through which one value could reach both) and an optional p_{bw} , `bandwidth_percentile` (default 68.27). For each vector x_i :

1. Find its k_{density} nearest neighbors (excluding itself), giving distances $d_{i,1}, \dots, d_{i,k}$.
2. $b_i = \text{percentile}_{p_{\text{bw}}}(d_{i,1}, \dots, d_{i,k})$ – the p_{bw} -th percentile of those distances (linear interpolation, via `calc_percentile_helper`), floored at a small epsilon (coincident neighbors at distance 0 would otherwise give $b_i = 0$, a zero-bandwidth Gaussian below).
3. $\text{density}(x_i) = \frac{1}{b_i^D} \sum_{j=1}^k \exp\left(-\frac{d_{i,j}^2}{2b_i^2}\right)$, where D is the ambient dimensionality.

This is an adaptive-bandwidth kernel density estimate: unlike a single dataset-wide radius, b_i shrinks in dense regions and grows in sparse ones, so the resulting density labels reflect *local*, not global, density. The $1/b_i^D$ normalization is not optional polish – without it, $d_{i,j}/b_i$ is scale-invariant, so a cluster and the same cluster uniformly stretched out score identically, which defeats the purpose of a *density* label.

p_{bw} is deliberately exposed as a tunable, undisguised heuristic, not presented as a calibrated standard deviation. The 68.27 default is anchored to the familiar 1-D-Gaussian fact “68.27% of the mass lies within one SD” – but that fact is 1-D-specific: for distances (norms) in D dimensions under an isotropic-Gaussian-scatter assumption, $\text{distance}/\sigma \sim \chi_D$, and the correct percentile for “one SD” shrinks with D (39.35% at $D = 2$, 19.88% at $D = 3$, ...). Rigorously correcting for this would require each point’s *local intrinsic dimension*, not the ambient D – and that is only available later, from `observable`’s per-ensemble SVD, not at the seeding stage. Using ambient D instead would actively over-correct for STC’s manifold-concentrated data (real local dimension is typically far below D). So p_{bw} stays a plain tunable parameter, not a disguised statistical estimate.

3.3.2 SKG `seeds`

Input:

- `data_vectors` - 2D real array

output:

- logical mask defining the selected seed points.

3.3.3 Algorithm

First, assign each $x_i \in \text{data_vectors}$ a density label via `density_labels`.

Sort the density labels descending. Start with the highest-density unvisited vector and mark it a seed. Its coverage radius is the *same computation* `calc_ensemble_growth_radius` uses for its own growth radius – a percentile (`exclusion_radius_percentile`, default 50.0, the median) of the distances to its own k_{density} nearest neighbors – called directly on the newly-selected seed with k_{density} in place of k_{min} , not a separate radius computed some other way. Mask every vector within that radius as visited. Continue with the next highest-density unvisited vector until none remain.

Tying the coverage radius to the same local-scale notion growth itself uses, rather than a single dataset-wide radius as an earlier draft of this algorithm did, is deliberate: a fixed global radius can suppress seed placement across a region much larger than what that seed’s own ensemble will ever actually grow into, leaving points “covered” by seed-exclusion but never reached by any grown ensemble – see `misc/STC-experiments/README.md` for a concrete demonstration of the resulting coverage gaps.

3.4 Ensemble identification

In parallel grow ensembles around each seed vector. Use OMP paradigms if and only if `do concurrent` parallelism is not supported due to external library calls (gfortran).

The SKG is called `ensemble_identification` and is an iteration wrapper for the below steps.

Consider each seed an ensemble of size one. Growing is done iteratively until stop conditions are reached. Each iteration has an ensemble $\mathcal{E}_t$, an 2D array of o observables $\mathcal{O}_{t-o+1}, \mathcal{O}_{t-o+2}, \dots, \mathcal{O}_t$.

3.4.1 First growth step

A seed is a single point: its `observable` would be the singular value decomposition of a one-column centered matrix, which is degenerate – no meaningful U , d , or G exists to compare against. `ensemble_identification` therefore performs its first `grow_ensemble + observable` call unconditionally, without invoking `accept_ensemble`; there is nothing yet to compare the result against. `accept_ensemble` is invoked starting from the second growth iteration onward, comparing iteration 2’s observable against iteration 1’s.

This bootstrap rule lives here, in the iteration wrapper, and not in `seeds`, `grow_ensemble`, `observable`, or `accept_ensemble` themselves. `seeds`’ purpose is identifying starting points, nothing else – and it runs *before* `calc_ensemble_growth_radius`, so it has no growth radius available to grow by even if it wanted to. Keeping `grow_ensemble`, `observable`, and `accept_ensemble` fully general and iteration-unaware keeps each independently testable in isolation. `ensemble_identification` is the one place that already knows which iteration it is, without introducing any new state elsewhere.

3.4.2 Stop Conditions

Growth for a single seed’s ensemble stops under exactly one of the following four conditions. `ensemble_identification` reports which one arose via an integer result-code output, `stop_reason` (see “Output” below) – a scheme distinct from, and orthogonal to, this codebase’s `ierr` error-code convention: all four conditions below are valid, non-error algorithmic outcomes, so `ierr` stays 0/success in every case.

1. **Maximum ensemble size reached.** The ensemble has grown to at least $f_{\max} \cdot N$ members, where N is the total number of input vectors and $f_{\max}$ is an optional input argument defaulting to 0.95. Growth reaching this size is treated as evidence against the hypothesis of intrinsic local structure (the ensemble has essentially absorbed the whole dataset), so **no ensemble is returned** for this seed – only the result-code flagging this outcome.
2. **Rejected after being stably accepted.** The ensemble has been accepted at least a times (an optional input argument, default 2) and the current growth step is rejected by `accept_ensemble`. The last *accepted* ensemble and its observable history are returned, together with the result-code for this condition.
3. **Rejected immediately.** The first time `accept_ensemble` is actually invoked (comparing iteration 2 against iteration 1 – see “First growth step” above) returns `.false.`, before the ensemble ever reached a accepted iterations. The bootstrap ensemble (iteration 1, always accepted unconditionally) is returned as the last accepted result, together with a result-code distinct from condition 2’s, so callers can tell a well-supported stop from one resting on a single, unconfirmed growth step.
4. **No further growth possible.** `grow_ensemble` returns the same membership it was given (no candidate vectors lie within the growth radius of any current member) – a natural fixed point. The last accepted ensemble and its observable history are returned, together with the result-code for this condition. This is the expected, “clean” way for growth to end.

Conditions 2-4 all return the last *accepted* ensemble; only condition 1 returns no ensemble at all. Evaluating condition 2 requires `ensemble_identification` to track a running count of how many growth iterations have been accepted in total for the current seed – a separate, cumulative counter, independent of the trailing o -window history below, since that window can be shorter than the true total once growth runs longer than o iterations.

3.4.3 Output

`ensemble_identification` returns, per seed:

- `final_ensemble_mask(n_vectors)`, logical – the accepted ensemble’s membership, using the same logical-mask convention as `seeds`. All `.false.` when `stop_reason` signals condition 1 (maximum ensemble size reached).
- `stop_reason`, integer – the result-code for whichever of the four “Stop Conditions” above applied.
- The trailing observable history, the last (up to) o iterations. Each $\mathcal{O}_t$ is not a flat vector – U is a $D \times d_t$ matrix whose width d_t can itself change between iterations, which is exactly what δ_d checks. Storing the **full** $D \times D$ decomposition per retained iteration, as already decided for `observable` above, sidesteps the variable-width problem: slice $U(:, 1:d_t)$ out on demand instead of needing variable-shape storage. Concretely:
 - `U_history(D, D, o)`, real;
 - `S_history(D, o)`, real – the singular values; eigenvalues, `normal_error`, and `tangent_scales` at any retained iteration are derived on demand from these plus `k_history` below, via the `normal_error/tangent_scales` SKGs, rather than stored redundantly;
 - `d_history(o)`, integer;
 - `G_history(o)`, real;
 - `mu_history(D, o)`, real;
 - `k_history(o)`, integer – ensemble size per retained iteration, needed to recover eigenvalues from singular values, $\lambda_j = s_j^2/(k-1)$;
 - `accepted_history(o)`, logical – whether the growth iteration retained in the corresponding column was accepted. Iteration 1 (the bootstrap step) is recorded as `.true.` by convention, even though `accept_ensemble` is never actually invoked for it (see “First growth step” above).
- `U_first(D, D)`, real, and `d_first`, integer – the tangent+normal basis and intrinsic dimension of iteration 1 (the bootstrap step), retained for the ensemble’s entire growth and never evicted, independently of the trailing o -window above. Distinct storage from `U_history/d_history` rather than widening those to $o+1$ or pinning one of their existing columns, so the trailing window keeps meaning exactly what it always has (o genuinely trailing iterations) and no existing caller’s interpretation of `U_history/d_history` changes. Set once, at iteration 1, alongside `low_confidence_mask` below (same call, no extra SVD). Used by `accept_ensemble`’s tangent-space-drift criterion (see “SKG `accept_ensemble`” below) to bound cumulative, not just step-to-step, rotation – see `misc/STC-experiments/README.md`’s “P5” discussion for the failure mode this closes: an ensemble that “walks” arbitrarily far in tangent-space orientation via many individually-small-enough steps, none of which alone would trip a step-to-step-only check.
- Optionally, `member_added_at_step(n_vectors)`, integer – 0 (or a documented sentinel) for non-members, the seed’s own designation for the seed itself, and the growth-iteration index at which each subsequent member joined otherwise. Answers “which members were added when” and “did this ensemble oscillate during growth” directly.
- Always collected, the same reasoning as `member_added_at_step` above (a byproduct of computation already happening unconditionally, not a separate cost to gate behind its own flag): `low_confidence_mask(n_vectors)`, logical – the membership from this seed’s iteration 1 (the unconditional bootstrap `grow_ensemble + observable` call, see “First growth step” above), reported *regardless* of what `stop_reason` the seed’s full growth eventually reaches, including seeds discarded entirely under condition 1 (maximum ensemble size reached), for which `final_ensemble_mask` above is all-`.false.`. Iteration 1 already has a genuine, non-degenerate `observable`/SVD behind it (unlike the single-point seed itself), so it is real ensemble data, not a guess – but it has never survived even one `accept_ensemble` comparison, hence “low confidence”: callers should treat it as a fallback, not a result on par with `final_ensemble_mask`. This is deliberately reported unconditionally per seed rather than the kernel itself deciding which points are “orphaned”: that decision belongs to whoever consumes `ensemble_masks` (e.g. LoManLe), by diffing a point against the *retained* ensembles – keeping `ensemble_identification` itself free of any opinion about what its caller does with an uncovered point.

3.4.3.1 Merged output

Suppose, Ensemble Identification found $N_{\mathcal{E}}$ ensembles (clusters).

The above per ensemble outputs are then collected into a structure of arrays representation of the output, where each of the per ensemble outputs is collected in an array with an extra trailing dimension of size $N_{\mathcal{E}}$, each index $i_{\mathcal{E}}$ along that dimension representing the data for ensemble number $i_{\mathcal{E}}$.

Thus the Ensemble Identification module’s main function returns the following two, three, or four dimensional Fortran arrays: each per-seed output above simply gains one extra trailing dimension of size $N_{\mathcal{E}}$ once merged. `U_history(D, D, o)` is already three dimensional per seed, so its merged form, `ensemble_U_history`, is Tensor Omics’ first four dimensional data structure: `ensemble_U_history(D, D, o, N_{\mathcal{E}})`. This is exactly where o earns its keep: D appears twice and $N_{\mathcal{E}}$ once more in that product, so keeping o small is what keeps this array’s memory footprint tractable.

- `logical :: ensemble_masks`
- `real :: ensemble_U_history`
- `real :: ensemble_S_history`
- `integer :: ensemble_d_history`
- `real :: ensemble_G_history`
- `real :: ensemble_mu_history`
- `integer :: ensemble_k_history`
- `logical :: ensemble_accepted_history`
- `real :: ensemble_U_first` – the merged form of `U_first` above, `ensemble_U_first(D, D, N_{\mathcal{E}})`.
- `integer :: ensemble_d_first` – the merged form of `d_first` above, `ensemble_d_first(N_{\mathcal{E}})`.
- `integer :: ensemble_stop_reason` – the result-code for whichever of the four “Stop Conditions” above applied to that ensemble, see “Output” above.
- Always collected (a byproduct of the per-seed growth loop, not a separate cost worth gating – see `member_added_at_step` above): `integer :: ensemble_member_added_at_step` – The integer values indicate at what iteration members were added. The row index in an ensemble’s `member_added_at_step` column vector refer to the `.true.` row indices in the respective ensemble’s `ensemble_masks` column vector.
- Always collected, same reasoning: `logical :: ensemble_low_confidence_masks` – the merged form of `low_confidence_mask` above, one column per seed regardless of `ensemble_stop_reason`.

3.4.4 Local Radius Identification

For growing ensembles we need a locally adapted radius with which at each step candidates for the addition are identified. Those candidates are vectors within the local ensemble specific radius.

The SKG should be called `calc_ensemble_growth_radius` or similar.

Use a fixed-count kNN pool and compute a percentile of the distances among a seed’s own $k_{\min}$ nearest neighbors, `radius_percentile` (0 to 100, default 50.0 – the median, this SKG’s original, non-parameterized behavior). Make $k_{\min}$ an optional argument with default value 30.

For each seed we store the growth radius in a 1D real array called `ensemble_growth_radii`.

`seeds` reuses this exact SKG for its own coverage radius (see “Seeding” above), called with k_{density} in place of $k_{\min}$ – not a second, separately-implemented percentile-of-k-NN-distances computation. `seeds` exposes its own call’s `radius_percentile` as `exclusion_radius_percentile`, independently of whatever percentile any other caller of this SKG uses for the actual growth-phase radius: a fixed, dataset-wide-in-spirit median exclusion radius was observed empirically to suppress seed placement across curvature extrema (peaks, troughs, kinks on a wavy manifold) that a seed’s own growth never actually reaches – see `misc/STC-experiments/README.md`. Shrinking `exclusion_radius_percentile` below 50 trades more, smaller ensembles for less over-eager suppression; it does not change the growth-phase radius itself.

3.4.5 Tangent Space Variant

This section describes the implementation used for the tangent space variant of STC.

Each iteration of `ensemble_identification` runs:

- `grow_ensemble`
- `observable`
- `accept`

3.4.5.1 SKG `grow_ensemble` Identify the candidate points within the input ensemble’s growth-radius $r_{\mathcal{E}}$:

$$\mathcal{C}_{\mathcal{E}_{t+1}} = \{x_k \in \text{data_vectors} \mid \|x_k - x_i\| \leq r_{\mathcal{E}} \exists x_i \in \mathcal{E}\}$$

3.4.5.2 SKG observable Calculate the tuple $\mathcal{O}_{\mathcal{E}_{t+1}} = (U_{\mathcal{E}_{t+1}}, d_{\mathcal{E}_{t+1}}, G_{\mathcal{E}_{t+1}}, \boldsymbol{\mu}_{\mathcal{E}_{t+1}}, \text{normal_error}_{\mathcal{E}_{t+1}}, \text{tangent_scales}_{\mathcal{E}_{t+1}})$ for the input ensemble $\mathcal{E}_{t+1}$, where

(1) $U_{\mathcal{E}_{t+1}}$, the tangent basis, is obtained from the **singular value decomposition** of the Ensemble’s centered member vectors – not from an eigendecomposition of their covariance (see “Numerical Linear Algebra” above for why). To obtain it, compute the neighborhood center

$$\boldsymbol{\mu}_{\mathcal{E}_{t+1}} = \frac{1}{|\mathcal{E}_{t+1}|} \sum_{x_j \in \mathcal{E}_{t+1}} \mathbf{x}_j,$$

construct the centered matrix

$$Y_{\mathcal{E}_{t+1}} = [\mathbf{x}_1 - \boldsymbol{\mu}_{\mathcal{E}_{t+1}}, \dots, \mathbf{x}_N - \boldsymbol{\mu}_{\mathcal{E}_{t+1}}], \text{ for } x_1, \dots, x_N \in \mathcal{E}_{t+1},$$

and its economy-mode singular value decomposition, using LAPACK `dgesdd` (`JOBZ='S'`), over the **full** set of D singular values/vectors, not only the top- d tangent ones:

$$Y_{\mathcal{E}_{t+1}} = U_{\mathcal{E}_{t+1}} S_{\mathcal{E}_{t+1}} V_{\mathcal{E}_{t+1}}^\top.$$

The retained “leftover” pairs are what make center, normal error, and tangent scales free below, and are also exactly what a later noise-tube covariance $\Sigma_\perp$ (needed once ensembles are reconciled) would be built from.

The corresponding eigenvalues are recovered from the singular values as

$$\lambda_j^{\mathcal{E}_{t+1}} = \frac{\left(s_j^{\mathcal{E}_{t+1}}\right)^2}{k_{\mathcal{E}_{t+1}} - 1}, \quad \lambda_1^{\mathcal{E}_{t+1}} \geq \lambda_2^{\mathcal{E}_{t+1}} \geq \dots \geq \lambda_D^{\mathcal{E}_{t+1}} \geq 0.$$

LAPACK SVD routines already return singular values in *descending* order (unlike `dsyev`’s ascending eigenvalues), so this matches the document’s own convention directly: the tangent basis and its eigenvalues are the *first* d columns/entries of `dgesdd`’s output, and the normal-space basis and eigenvalues are the remaining $D - d$.

(1b) $\boldsymbol{\mu}_{\mathcal{E}_{t+1}}$, the ensemble center, is simply the mean vector already computed above to form $Y_{\mathcal{E}_{t+1}}$ – no further work needed.

(1c) $\text{normal_error}_{\mathcal{E}_{t+1}}$ and $\text{tangent_scales}_{\mathcal{E}_{t+1}}$ are obtained from the retained eigenvalues via the **normal_error** and **tangent_scales** SKGs defined below.

(2) The Spectral Gap is calculated as follows.

For every candidate intrinsic dimension

$$r = 1, \dots, D - 1$$

compute its Spectral Gap

$$G(r) = \frac{\lambda_r^{\mathcal{E}_{t+1}}}{\lambda_{r+1}^{\mathcal{E}_{t+1}} + \epsilon},$$

where ϵ is a minimal positive real number to avoid division by zero.

(3) and the estimated intrinsic dimension

$$d_{\mathcal{E}_{t+1}} = \arg \max_r G(r)$$

and set the current ensemble’s spectral gap to

$$G_{\mathcal{E}_{t+1}} = G(d_{\mathcal{E}_{t+1}})$$

Note that an optional argument o with default value of e.g. 10 indicates how many observables of the last o iterations are stored. Thus, each ensemble $\mathcal{E}_{t+1}$ has a two dimensional real array of observables, one per column:

$$\Omega_{\mathcal{E}_{t+1}} = [\mathcal{O}_{\mathcal{E}_{t-o+2}}, \mathcal{O}_{\mathcal{E}_{t-o+3}}, \dots, \mathcal{O}_{\mathcal{E}_{t+1}}]$$

In addition to this trailing window, $\mathcal{O}_{\mathcal{E}_1}$ (iteration 1, the bootstrap observable) is *always* retained as **U_first/d_first** (see “Output” above), independently of o ’s FIFO eviction. Together, $\Omega_{\mathcal{E}_{t+1}}$ ’s trailing window and $\mathcal{O}_{\mathcal{E}_1}$ form the reference set **accept_ensemble**’s tangent-space-drift criterion compares the new candidate against – see “SKG **accept_ensemble**” below. For $t+1 \leq o$, $\mathcal{O}_{\mathcal{E}_1}$ already sits in the trailing window too ($U_{\text{history}}(:, :, 1)$); comparing the candidate against it twice (once via **U_first**, once via the window) is harmless – it does not change a max – and simpler than special-casing the overlap away.

3.4.5.3 SKG normal_error Given intrinsic dimension $d_{\mathcal{E}}$ and the eigenvalues $\lambda_1^{\mathcal{E}} \geq \dots \geq \lambda_D^{\mathcal{E}}$ from **observable**, the mean squared residual of the ensemble’s members off its $d_{\mathcal{E}}$ -dimensional tangent subspace is

$$\text{normal_error}_{\mathcal{E}} = \sum_{j=d_{\mathcal{E}}+1}^D \lambda_j^{\mathcal{E}}.$$

No pass over the ensemble’s member vectors is required; the sum is already implied by the singular value decomposition computed in **observable**.

3.4.5.4 SKG tangent_scales Given the same $d_{\mathcal{E}}$ and eigenvalues, the extent along each tangent direction is

$$\text{tangent_scales}_{\mathcal{E}}(j) = \sqrt{\lambda_j^{\mathcal{E}}}, \quad j = 1, \dots, d_{\mathcal{E}}.$$

3.4.5.5 SKG accept_ensemble This routine returns a Boolean indicating whether the grown ensemble $\mathcal{E}_{t+1}$ is accepted or not.

A grown ensemble ($\mathcal{E}_{t+1}$) is assessed against a **reference set** $\mathcal{R}$ of previously accepted observables, not merely against $\mathcal{O}_{\mathcal{E}_t}$ alone:

$$\mathcal{R} = \{\mathcal{O}_{\mathcal{E}_1}\} \cup \{\mathcal{O}_{\mathcal{E}_{t-o+2}}, \dots, \mathcal{O}_{\mathcal{E}_t}\}$$

i.e. the bootstrap observable (**U_first/d_first**, always retained, see “Output” above) plus the trailing o -window. Comparing against this whole set, not just $\mathcal{O}_{\mathcal{E}_t}$, is what closes the “no cumulative-rotation budget” gap: a step-to-step-only check cannot see an ensemble that walks arbitrarily far in tangent-space orientation via many individually-small steps, since each individual step still passes; anchoring every candidate against the ensemble’s own original state (and its recent history) bounds the *cumulative* drift instead.

$\mathcal{E}_{t+1}$ is accepted if all four of the following hold:

(1) Tangent-space drift. For every reference $\mathcal{O}_{\mathcal{E}_r} \in \mathcal{R}$ with $d_r = d_{\mathcal{E}_{t+1}}$ (references with a different d_r are skipped for this criterion – there is no shared tangent dimension to compare angles over, judged instead by criterion (2)), compute the principal angles $\theta_1, \dots, \theta_{d_r}$ between U_r and $U_{\mathcal{E}_{t+1}}$ via **dgesvd** on $M = U_r^{\top} U_{\mathcal{E}_{t+1}}$, whose singular values are $\cos \theta_k$ directly (see “Numerical Linear Algebra” above), then the **chordal distance**

$$\mathcal{C}(U_r, U_{\mathcal{E}_{t+1}}) = \sqrt{\sum_{k=1}^{d_r} \sin^2 \theta_k}$$

(the standard Grassmannian chordal/projection-Frobenius distance – Edelman, Arias & Smith 1998; related to the Davis-Kahan $\sin \Theta$ theorem). **Accept** requires

$$\max_{r: d_r = d_{\mathcal{E}_{t+1}}} \mathcal{C}(U_r, U_{\mathcal{E}_{t+1}}) \leq \text{chordal_dist_max_as_prcnt_of_range} \cdot \sqrt{d_{\mathcal{E}_{t+1}}}$$

(vacuously satisfied if no reference shares $d_{\mathcal{E}_{t+1}}$). **chordal_dist_max_as_prCNT_of_range** (optional argument, 0 to 1, default 0.5) replaces the earlier **alpha_max** (a raw angle in radians): $\mathcal{C}$ ’s range is $[0, \sqrt{d}]$, so this parameter

states the tolerated fraction of that range directly, without an intermediate angle-to-sine conversion, and without needing a separate $\sqrt{d}$ -dependent knob per intrinsic dimension – the $\sqrt{d}$ scaling is handled internally. It reduces to a familiar special case at $d = 1$: $C = |\sin \theta|$, so `chordal_dist_max_as_prct_of_range` = $\sin(\alpha_{\max})$ recovers exactly the old single-angle boundary (default $0.5 \approx \sin(30^\circ)$), a literal carry-over of the old default under this substitution – provisional, like every other default in this family, pending empirical re-tuning once this criterion runs on real data).

Only the candidate is compared against each reference – **not** every pair within $\mathcal{R}$ itself ($\binom{|\mathcal{R}|}{2}$ pairs). This is sufficient, not a shortcut: every reference in $\mathcal{R}$ already passed this same criterion against every *other* reference present in $\mathcal{R}$ at the growth step it was itself accepted (an inductive invariant – FIFO eviction only ever removes references from $\mathcal{R}$, which cannot break a “already mutually within tolerance” property among the ones that remain). So checking the one new candidate against the (at most) $o + 1$ existing references costs $O(o)$ small SVDs per growth step, not $O(o^2)$.

(2) Intrinsic-dimension drift. Both

$$d_{\text{to_first}} = |d_{\mathcal{E}_{t+1}} - d_{\mathcal{E}_1}| \leq d_{\max} \quad \text{and} \quad d_{\text{to_last}} = |d_{\mathcal{E}_{t+1}} - d_{\mathcal{E}_t}| \leq d_{\max}$$

must hold – i.e. $\max(d_{\text{to_first}}, d_{\text{to_last}}) \leq d_{\max}$. Compared only against the first and the immediately preceding iteration (not the full o -window, unlike criterion (1)) – cheap, and closes the same first-vs-cumulative gap for d that criterion (1) closes for orientation, without needing d history beyond what `d_first`/the previous iteration’s d already provide.

(3) Spectral-gap drift.

$$\left| \log \frac{G_{\mathcal{E}_{t+1}}}{G_{\mathcal{E}_t}} \right| \leq G_{\max}$$

Consecutive only (t vs. $t + 1$), unchanged from before. G is a magnitude (the ratio of two adjacent eigenvalues), not an orientation – the “walk arbitrarily far while every single step looks fine” failure mode criterion (1) fixes is specific to direction, not scale, so this criterion is left as is for now.

(4) Residual (RMSE) drift. Let $\text{RMSE}_{\mathcal{E}} = \sqrt{\text{normal_error}_{\mathcal{E}} + \epsilon}$ (the root of the `normal_error` SKG’s output, itself already free – see “SKG `normal_error`” above; no new SVD or storage, `normal_error` at any retained iteration is already derivable on demand from `S_history`/`k_history`), where ϵ is the same minimal-positive-real convention `observable`’s own spectral-gap formula already uses (`epsilon(1.0_real64)` in Fortran) – unlike G , which is a *ratio* already protected by its own $+\epsilon$ denominator, `normal_error` is a raw sum of eigenvalues and can be genuinely, exactly zero (a perfectly flat/collinear ensemble, no noise at all in the normal directions); without this guard, $\log(\text{RMSE}_{t+1}/\text{RMSE}_t)$ is $\log(0/0)$, undefined, the first time growth is ever perfectly noise-free. Accept requires

$$\left| \log \frac{\text{RMSE}_{\mathcal{E}_{t+1}}}{\text{RMSE}_{\mathcal{E}_t}} \right| \leq \text{RMSE_change_max}$$

(optional argument, default $|\log(1.5)| \approx 0.405$), consecutive only, same structural form as criterion (3). G (a ratio between adjacent eigenvalues) and RMSE (the absolute magnitude of *all* residual eigenvalues combined) are genuinely different signals, not a restatement of the same information under a different name: an ensemble can hold G essentially constant across growth (its dimensionality estimate stays well-supported) while the absolute off-tangent-subspace spread still creeps upward – criterion (3) alone cannot see that, since it only ever compares λ_d against its immediate neighbor λ_{d+1} , never the total normal-space spread. This criterion is what catches that.

3.5 Ensemble Reconciliation

This step implements a final polishing, quality-filtering, and conflict resolution step of the identified ensembles. In the tangent-space-variant intersections between ensembles are identified and reported. Note that by default ensembles are not merged based on intersections, because the intersections are needed by our Manifold Learning algorithm to stith together local linear manifolds.

The SKG `ensemble_reconciliation` is a thin, two-call orchestrator over two sibling SKGs, each independently testable and independently reusable: first `filter_ensembles` (which ensembles are even eligible to contribute a pair, judged purely per-ensemble – see “Filtering ensembles before merging” below), then this module’s own `merge_to_super_ensembles` (the actual pairwise-intersection/grouping decision, over eligible ensembles only). Conflating eligibility and merging into one procedure would mean every new filtering criterion has to touch the same monolithic merge logic; keeping them separate means a new per-ensemble criterion is *only* ever a new function in `filter_ensembles`, never a change to how merging itself works. `merge_to_super_ensembles` is described first

below, since it is what most of this section’s own algorithm description concerns; “Filtering ensembles before merging” covers the `filter_ensembles` half.

`merge_to_super_ensembles` first identifies ensembles that intersect. It then offers three different modes of how to process intersections: * (1) just report them * (2) merge with a minimal set Overlap Coefficient (SOC; default 0.9) * (3) merge all, i.e. if the intersection has size ≥ 1 , merge.

The module does not alter the result produced by “Ensemble Identification”. It creates a sparse representation of an adjacency matrix of edges between ensembles indicating which have intersections (mode 1), or have been merged (modes 2 and 3).

Detecting an intersection at all already requires computing $|\mathcal{E}_i \cap \mathcal{E}_j|$, needed by every mode, and each ensemble’s own size $|\mathcal{E}_i|$ is cheap to precompute once per ensemble ($O(N_{\mathcal{E}})$ total, not once per pair). The **Overlap Coefficient** (Szymkiewicz-Simpson coefficient) itself,

$$\text{OC}(\mathcal{E}_i, \mathcal{E}_j) = \frac{|\mathcal{E}_i \cap \mathcal{E}_j|}{\min(|\mathcal{E}_i|, |\mathcal{E}_j|)},$$

is then a single extra $O(1)$ step per pair – even cheaper than the Jaccard Similarity Index this replaces, since the denominator is just the smaller of the two already-precomputed sizes, not a union requiring both sizes and the intersection combined. Overlap Coefficient (unlike Jaccard) is exactly the statistic for “is one ensemble, or most of it, contained within the other,” which is the actual question a true or partial subset relationship poses: two ensembles where the smaller is fully contained in the larger score $\text{OC} = 1$ regardless of how much larger the other one is, whereas their JSI would be driven arbitrarily low by the size difference alone – exactly the containment relation JSI fails to capture. Reporting it is made available regardless of mode, via an optional user flag, rather than tied to mode 2 specifically. Modes 1 and 3 do not need it for their own decision, so **the implementation must guard the computation behind that flag** (an `if (report_overlap_coefficient) ...`, not an unconditional computation): the added cost is small, but it is not exactly zero, and nothing here should be computed that nobody asked for.

The output assumes a maximum number of ensembles that can belong to one merged group, to enable allocation of the sparse adjacency matrix representation. By default this max number is $\min(1024, N_{\mathcal{E}})$. Thus the output can be represented as the two dimensional `integer(int32) :: super_ensembles` array in which each column contains one “super-ensemble”, i.e. a group of ensembles that have intersections (mode 1) or have been merged (modes 2 and 3); its rows hold that group’s member ensembles, padded with 0 (an invalid ensemble number, since ensembles are 1-indexed) beyond the group’s actual size. The content of each cell are the column indices of module Ensemble Identification’s `ensemble_masks` output, thus identifying the ensembles directly via their indices. If Overlap Coefficient reporting was requested, an optional two dimensional real `super_ensembles_overlap_coefficient` array is returned additionally. It has the same number of columns as `super_ensembles` and one row less. Each column in `super_ensembles_overlap_coefficient` represents the super-ensemble in `super_ensembles` directly. Within each column l , each cell index c_i indicates the Overlap Coefficient between the ensemble number stored in `super_ensembles(c_i, 1)` and `super_ensembles(c_i + 1, 1)`.

3.5.1 Filtering ensembles before merging

`filter_ensembles` decides which ensembles are even eligible to contribute a pair to `merge_to_super_ensembles`, from three independent, per-ensemble criteria, each its own small SKG returning its own `eligible(N_E)` mask, combined by `filter_ensembles` itself via a plain logical AND. A criterion whose own threshold/allowed-set argument is omitted contributes an all-`true`. mask (no constraint from that criterion), so omitting every optional argument makes `filter_ensembles` itself a true no-op (every ensemble eligible).

- **filter_ensembles_by_stop_condition**: an optional `allowed_stop_reasons` (a 4-entry logical array, one flag per `ensemble_identification` Stop Condition) excludes ensembles by Stop Condition – e.g. excluding `rejected_immediately` (a single, unconfirmed growth step, the lowest-confidence outcome) keeps low-confidence ensembles from ever being merged.
- **filter_ensembles_by_dimension**: optional `filter_dim_min`/`filter_dim_max` (each independently optional, both inclusive) bound an ensemble’s *final* intrinsic dimension – see `ensemble_final_observable` below for what “final” means here. Named `dim`, not `d`, specifically to stay distinct from `accept_ensemble`’s own `d_max` (see criterion (2) there): that one bounds a *difference* of consecutive/reference dimension estimates during

growth, this one bounds the *value* of the final dimension itself, after growth has already terminated – two unrelated parameters that happened to share a name before this distinction was introduced.

- **filter_ensembles_by_var_explained**: an optional `var_explained_min` bounds an ensemble’s *final* classical variance explained, $\sum_{j=1}^d \lambda_j / (\sum_{j=1}^d \lambda_j + \text{normal_error})$ – the familiar PCA energy-ratio (a scale/sqrt-based alternative, $\sum_j s_j / (\sum_j s_j + \text{RMSE})$, was considered during this criterion’s own design and rejected in favor of the classical, squared-units form specifically for being the measure data scientists already expect, even though the scale-based alternative would likely be somewhat more outlier-robust). Motivation: some ensembles “learn noise” – grow into a region where the tangent-subspace fit is technically accepted by `accept_ensemble`’s own criteria but explains little of the ensemble’s own spread – and this criterion lets such ensembles be excluded from merging without discarding them from `ensemble_identification`’s own output.

All three criteria need each ensemble’s *final* accepted state (d, singular values, size) – not simply the last *populated* history column, for the same reason `misc/mod_STC.md`’s own “Ensemble identification” section describes for `U_first/history` in general (a rejected final candidate can sit in the last populated column). This extraction is itself a small, shared SKG, `ensemble_final_observable` (see “Implementation Modules” above for where it lives and why) – `filter_ensembles` calls it once, up front, rather than each of its three sub-filters (or, worse, some external caller) re-deriving it independently.

An ineligible ensemble’s pairs are simply never considered by `merge_to_super_ensembles` – no array copying or compaction, an `eligible(i) .and. eligible(j)` guard ANDed into the existing per-pair decision, the same “logical AND of masks” shape mode `MODE_MERGE_OVERLAP_COEFFICIENT`’s own Overlap Coefficient threshold check already uses. Excluded ensembles are unaffected everywhere else – they keep existing in `ensemble_identification`’s own output, in every point/CSV/JSON field this whole family produces, and in `tox_stc_json`’s own `overlap_coefficient_matrix` (computed independently of this module, see `tox_stc_json.F90`), which applies the identical mask for the same reason: so the full pairwise matrix stays consistent with whatever `ensemble_reconciliation` actually merged. Only `merge_to_super_ensembles`’s own pairing/grouping decision is affected – an excluded ensemble simply never appears in any `super_ensembles` group or in that matrix. `ensemble_reconciliation` publishes the combined mask *and* the three individual per-criterion masks as its own output, so a report (see “Scientific plots / Visualization” below) can show *which* criterion excluded a given ensemble, not merely that one did.

3.6 Estimate parameters from data

The crucial parameters are `k_min`, `k_density`, `density_quantile`, `chordal_dist_max_as_prct_of_range`, `G_max`, `d_max` (`RMSE_change_max`, `accept_ensemble`’s fourth criterion, is not estimated by this SKG – see “SKG estimate_stc_parameters” below). Grid-searching them, or the bootstrap-resampling scheme considered and rejected during this section’s design (per-seed resampled SVDs to measure noise-only wobble in angle/gap), both cost far more than the rest of the pipeline combined. This SKG estimates all six from one cheap, coarse pass instead: grow a handful of anchors into rough local neighborhoods using the same primitives (`density_labels`, `observable`) the real pipeline already has, then read the parameters off simple summary statistics of that pass. It is a **separate, optional** step – it neither runs automatically as part of `seeds` or `ensemble_identification`, nor requires that either has already run; a caller may run it, inspect the proposed values, adjust them, and only then invoke the real pipeline, or skip it entirely and supply `k_min/chordal_dist_max_as_prct_of_range/etc.` by hand as always.

3.6.1 SKG sample_estimator_anchors

Receives `density_labels`’ own output (see “Seeding” above – this SKG does not recompute density itself) and an optional `n_anchors` (default 5). Sort the vectors by density label and pick `n_anchors` of them, by nearest-rank (not interpolated – these must be genuine point indices), at the percentiles $\frac{100}{n_{\text{anchors}}}, \frac{200}{n_{\text{anchors}}}, \dots, 100$ of that sorted order – e.g. `n_anchors = 5` gives 20/40/60/80/100%ile, generalizing the original “one point at each 20%ile” choice to a tunable anchor count. We call the resulting points the estimator anchors (EAs).

3.6.2 SKG grow_estimator_anchor_clouds

Each EA starts as its own single-point cloud. Growth proceeds in rounds: every EA proposes the single closest not-yet-claimed point to *any* member of its own current cloud (not just to the anchor itself – the cloud grows as a region, the same way `grow_ensemble` grows an ensemble); among all pending proposals, the closest one is claimed by its proposing EA, ties broken by which EA is nearer. This is a multi-source region-growing process – equivalent to

a multi-source Prim’s/Dijkstra expansion, one shared frontier across all EAs, distance-ordered claims – not n_{anchors} independent kNN queries: a point equidistant-ish between two EAs must go to whichever is *currently* closer, which can only be answered by comparing proposals across EAs at claim time, not by asking each EA in isolation.

Implemented as a brute-force rescan, not via the k-d tree: every round, for every EA, scan all not-yet-claimed points against all of that EA’s current cloud members directly on **vectors**, no tree traversal. **f42_kd_tree**’s own primitives answer “nearest point(s) to one query point,” not “nearest unclaimed point to any member of a growing region,” so using it here would mean re-deriving that query shape from scratch – more implementation complexity for a saving that does not matter at this SKG’s intended scale. With n_{anchors} small (default 5) and total growth capped small by **seed_max_set_size** (default 5% of N) precisely so that clouds stay local (see below), the whole rescan is $O(\text{rounds} \times n_{\text{anchors}} \times N \times \text{cloud size})$, all of them small factors by construction – trivial in absolute terms for the datasets this SKG targets. This is a deliberate simplicity-over-asymptotic-elegance trade, consistent with this whole SKG’s “minimal complexity” mandate; it is not the right choice if **seed_max_set_size** is ever pushed towards its unbounded (100%) extreme on a large N , where the original per-EA-partitions-everything draft’s own cost concern (see “Growth stops...” below) returns.

Growth stops when either no unclaimed point remains reachable, or the total number of claimed points across all EAs reaches an optional **seed_max_set_size** (0 to 100, default 5.0) percent of N . The default is deliberately small: growing until the *entire* dataset is partitioned among n_{anchors} anchors (the original, unbounded draft of this algorithm) gives each EA a cloud averaging N/n_{anchors} points – for $n_{\text{anchors}} = 5$, a fifth of the whole dataset, easily spanning a kink or bend and no longer “local” in any sense **observable**’s SVD could meaningfully summarize. Bounding total growth to a small percentage of N keeps every EA’s cloud a genuinely local neighborhood, at the cost of not necessarily reaching every point – acceptable here, since covering the dataset is **seeds**’ job, not this SKG’s.

3.6.3 SKG estimate_stc_parameters

The orchestrator. Runs **density_labels**, then **sample_estimator_anchors**, then **grow_estimator_anchor_clouds**, then **observable** once per EA on its final cloud (reused directly – this SKG does not reimplement the SVD). From the n_{anchors} per-EA results $(k_i, \bar{d}_i^{\text{median}}, d_i, G_i)_{i=1}^{n_{\text{anchors}}}$, where k_i is EA i ’s final cloud size and $\bar{d}_i^{\text{median}}$ the median distance from EA i ’s anchor to its own cloud members:

- $k_{\min} \leftarrow \text{median}_i(k_i)$
- $k_{\text{density}} \leftarrow k_{\min}$ – reused, not separately estimated, the same relationship **density_labels**’ own default already has to **calc_ensemble_growth_radius**’s $k_{\min}$ (see “Seeding” above).
- $\text{density_quantile} \leftarrow \text{median}_i(\bar{d}_i^{\text{median}})$ – a literal radius (data units), used directly wherever a radius is needed, not converted into a 0-100 percentile: **seeds/ calc_ensemble_growth_radius**’s own **exclusion_radius_percentile/ radius_percentile** already exist to *choose* a percentile of a kNN pool; this SKG instead hands back an already-measured absolute scale, which a caller may use directly as a growth or exclusion radius.
- For every pair $i < j$ among the EAs with $d_{ij} = \min(d_i, d_j) > 0$, compute the principal angles $\theta_1, \dots, \theta_{d_{ij}}$ between U_i and U_j over their shared rank d_{ij} – the same **dgesvd-on- $U_i^T U_j$** machinery **accept_ensemble** uses (see “Numerical Linear Algebra” above), but *not* gated on $d_i = d_j$ the way **accept_ensemble** gates its own criterion (1) (there, mismatched- d references are vacuously skipped, judged instead by criterion (2)); here, with only $\binom{n_{\text{anchors}}}{2}$ pairs total (10 at the default $n_{\text{anchors}} = 5$), skipping mismatched pairs risks too few (or zero) samples to estimate a quartile from at all. Comparing over the shared rank always produces a value, at the cost of diverging slightly from **accept_ensemble**’s own convention – a deliberate trade for this SKG’s specific purpose, not an oversight. Pairs with $d_{ij} = 0$ (at least one EA has no meaningful tangent direction at all) are excluded from the sample entirely, the same “nothing to compare” convention as **accept_ensemble**’s own $d_r > 0$ guard.
- Chordal distance per pair, normalized by its own $\sqrt{d_{ij}}$ so pairs of differing rank are directly comparable – exactly **chordal_dist_max_as_prcnt_of_range**’s own definition (see “SKG **accept_ensemble**” above), applied here to EA pairs instead of candidate-vs-reference pairs:

$$c_{ij} = \mathcal{C}(U_i, U_j) / \sqrt{d_{ij}} = \sqrt{\sum_{k=1}^{d_{ij}} \sin^2 \theta_k} / \sqrt{d_{ij}}$$

- $\text{chordal_dist_max_as_prcnt_of_range} \leftarrow Q_p(\{c_{ij}\}_{i < j})$
- $G_{\max} \leftarrow Q_p(\{|\log(G_i/G_j)|\}_{i < j})$
- $d_{\max} \leftarrow Q_p(\{|d_i - d_j|\}_{i < j})$

where Q_p is the p -th percentile (linear interpolation, via `calc_percentile_helper`) and p , `first_quartile_percentile`, is itself an optional argument (default 25.0, the first quartile) – not hardcoded, following the same “expose the heuristic, do not disguise it” precedent as `bandwidth_percentile/exclusion_radius_percentile` above.

This whole procedure is a heuristic starting point, not a converged answer: $n_{\text{anchors}} = 5$ (10 pairs) is a small sample, anchors are chosen by density quantile alone (no spatial-spread guarantee – on a dataset with strong density heterogeneity, all anchors could land in similar regions), and `grow_estimator_anchor_clouds` has no curvature-awareness of its own, only the `seed_max_set_size` size cap. Treat its output the way any of this family’s other tunable defaults are treated: a reasonable value to start from and refine, not a guarantee.

4 Command line interface (CLI) in C

Implemented in `C-layer/ (csv_table.h/csv_table.c, stc_cli.c`; see `C-layer/README.md` for usage) – not `src/f42/`, since it is C, not Fortran, and not part of the generator’s own kernel/wrapper split.

Use GNU `argp` to create a C command line interface that leverages `libcsv` to parse an input table with callbacks to create a 2D real Fortran array of input data vectors, then call the Fortran `_c` API to run our Shape Truthful Clustering from the command line interface.

4.1 Real 2D array from a CSV with libcsv

Write a F42 module that uses C’s `libcsv` to parse a CSV and creates a transposed 2D array from the input data. In this first version assume all values are real numbers. Also note our `src/f42/f42_safeguard.F90` ensuring that C types match Fortran types.

The module expects the following arguments: `* file`, i.e. the complete file path to the input CSV `* separator`, i.e. the char used to separate columns `* header`, i.e. a flag indicating whether a header line is present

The module implements the following parsing approach: `* Query libcsv for the number of columns`, e.g. by parsing the first line, then instantiate a real 2D array with `nrows` equal to the number of columns. If possible use some file information to get the number of lines and instantiate the the read 2D array with `nrows` and `nlines`, i.e. the input CSV will be stored transposed in Fortran, because Fortan is column major. `* If not possible to obtain the number of lines`, makes this a required input argument. `* Then use libcsv’s callback mechanism to populate the above output 2D array`. Do all of this still in C. `* Once parsing has passed provide a pointer and dimension information to pass the 2D array with zero copy to Fortran`. Our codebase has many examples of how to do this with zero copy.

4.2 Arguments of CLI

The command line interface will require arguments: `* file`, i.e. the complete file path to the input CSV `* separator`, i.e. the char used to separate columns `* header`, i.e. a flag indicating whether a header line is present `* the arguments required by STC`, like `k_min`, etc. `* include optional arguments with default values`, i.e. the user can provide values but is not required to. `* output-dir`, required argument of a directory-path to which to write our output files, including the HTML/D3 plot.

5 Scientific plots / Visualization

Each run of STC shall produce a scientific plot and result files into an output directory. The plot is implemented using HTML,CSS,JavaScript, and especially D3. The template for visualizatin is static and only the data is included in the HTML code in the form of serialized JSON strings.

The required third party libraries are included as static strings in dedicated Fortran classes in order to ensure that compiled shared objects will have no trouble finding those third party sources. See `@helper/embed_stc_html_assets.py` for how to generate those static assets. The current plotting is implemented in `@src/tox_stc_json.F90`.

5.1 2D Plot definitions

This entire section focusses on two dimensional plots and is used only for STC applied to 2D input data.

Each single STC-run will be presented in one self contained interactive scientific plot. The actual plot shall take approximately 80% of the window size to ensure it is the central object of attention. There is a hamburger menu on the top left corner, which by default is opened. The -key toggles the menu. In the menu all control elements for the interactive plot are contained.

The actual plot has a little export icon in the top right corner, enabling the user to export the current visible state to SVG and using an adequate JS-library like html-to-image allows export to PNG or JPG.

The control elements are listed by plot attributes — see below.

5.1.1 Colors

In general the color grey will be used to indicate points and seeds that were not assigned to any ensemble.

5.1.2 Points

5.1.2.1 Point color The point color can be linked to data properties. Four modes are offered:

- **Intersection density (default):** a heatmap scale from low (blue) to high (red) number of ensembles a point has been assigned to. A continuous color scale legend is displayed alongside the plot, mapping values to colors – not just a couple of discrete example swatches.
- **Ensemble ID:** the user can select this option to associate color with ensemble ID. As points can have various associations with ensembles, the largest ensemble is chosen to determine the point’s color. If this option is chosen, a new multi-select menu appears that allows the user to select which ensembles are displayed by point-color.
- **Super-ensemble ID:** a toggle enables the user to select point color to be associated not with ensemble-IDs but with super-ensemble-IDs. The same color controls (largest-wins, multi-select) apply here, too.
- **Low-confidence fallback coverage:** colors each point by whether it belongs to a retained ensemble (`final_ensemble_mask`), only to a low-confidence iteration-1 fallback (`low_confidence_mask`, see “Output” above) and no retained ensemble, or to neither. A short caption/title is shown in this mode’s own legend explaining these three categories, since the distinction is not otherwise self-explanatory from the color alone.

5.1.2.2 Point shape The default point shape are filled circles. The diameter can be adjusted, i.e. increased or decreased with a slide control.

20% larger triangular shapes indicate seeds. This can be toggled on and off. The triangle color can be linked to the point color or not (toggle control element). If not associated with ensemble color, the triangles are shown in black.

5.1.2.3 Mouse over information Upon hovering over a point a small transparent pop-up appears displaying the below information:

- The coordinates of the data point.
- The ensemble-IDs the point has been assigned to, and for each the point’s residual length and the ensemble’s stop condition.
- The super-ensemble-IDs the point has been assigned to, and for each the ensemble-IDs that super-ensemble contains.

5.1.3 Lines - Local tangent representation

Each ensemble has a final tangent space. In the case of 2D input this will be a line in 2D. These lines are drawn within the limitations of their respective ensemble’s range. To infer this, the ensembles elements are projected onto the line and the line is drawn between the two most extreme points. The calculations of the start and stop points are done in Fortran within the visualization module and exported to JSON.

Upon hovering over a line the ensemble ID and the singular value is shown.

5.1.3.1 Line Color The line color matches the ensemble color - see point color. A multi-select menu allows toggling ensemble tangent lines on and off.

A control element allows the user to select a single color for all lines.

Line thickness can be adjusted with a slider control element.

5.1.4 Seed growth radii

The seed growth radii are displayed as transparent circles. The default color is the same as the triangles'. The user can toggle, whether these circle's are assigned the color of their respective ensemble they grew into (toggle). The transparency can be adjusted with a slider control element.

5.2 Additional plots

Additional plots are available to the user. They can be toggled by dedicated buttons in a top horizontal menu bar. By default all additional plots are toggled off. If switched on those plots are shown in separate `<div/>` elements positioned vertically below the main plot.

5.2.1 Ensemble Overlap Coefficient (OvC) Heatmap

Ensemble IDs are sorted ascending and an upper triangular heatmap, excluding the diagonal, is rendered showing in a heat scale from zero (blue) to 1 (red) is drawn. A vertical scale at the right of the plot visually maps OvC values to their respective color. The ensemble IDs are used as heatmap axes labels. By default both axes show the ensemble IDs. Rows and Columns are surrounded by a thin black line to ease finding the cells of interest. Hovering over a cell with the mouse displays the ensemble IDs and their respective OvC value. The plot has its own hamburger menu. In it the user can toggle x and y axes labels on and off, and toggle the row and column surrounding lines.

5.2.2 Super-Ensemble Tree

An interactive tree is rendered showing which super-ensembles contain which ensembles. The color of the contained ensembles is identical to the point color in the main plot - referring to the point color in case it is associated with ensemble IDs. Clicking on a super-ensemble inner node, collapses all descendent nodes (ensemble leaf nodes). The shape of ensemble leaf nodes indicate stop conditions. Hovering over a super-ensemble node shows the number of contained ensembles, the total number of data-points and the fraction of contained data-points relative to the total number of input data points (ambient vectors). Hovering over an ensemble leaf node shows the below information:

- The ensemble-IDs
- The ensemble's stop condition
- The ensemble's seed
- The ensemble's final observable vector
- The ensemble's number of contained points and the fraction of the total input data these points form.

5.2.3 Ensemble Observable Plots

This is a combination of three standard two dimensional line-plots, including the data-points connected by line-segments (like R's `plot(..., type="both")`).

At the top horizontal center of the plot the user has a selection-menu-item with which they can select the ensemble for which to plot the below observable plots. By default no ensemble is selected and "all ensembles" is shown.

Note that these Ensemble Observable Plots are only applicable to ensembles that actually have a $\Omega_{\mathcal{E}}$ associated with them. This implies that ensembles that are rejected in the first growth step will not be eligible for these plots.

5.2.3.1 Ensemble specific Observable Plots In the case the above selection-menu-item has a specific Ensemble $\mathcal{E}_T$ selected, the below defined plots are generated.

Three observable plots are shown side by side, or, if the window has not enough width, automatically flow to vertical ordering.

Each plot shows the development of one observable over the last o iterations, including the first at $t = 1$ – except the Consecutive Tangent-Space Drift plot, which has no point at $t = 1$; see below. Thus the x-axis indicates iteration number (time). Note that it is not in scale, as the iteration number are $1, \dots, T - o, T - o + 1, \dots, T$. Three dots between 1 and $T - o$ indicate this. Also no segment is drawn between the point at x-coordinate 1 and the subsequent one.

Such a line-plot is generated for:

- the Spectral Gap $\mathcal{G}_{\mathcal{E}_{t_i}}$. Hovering over a point $i > 1$ here shows the Spectral-Gap drift $|\log \frac{\mathcal{G}_{\mathcal{E}_{t_i}}}{\mathcal{G}_{\mathcal{E}_{t_{i-1}}}}|$
- the RMSE, i.e. $\text{RMSE}_{\mathcal{E}_{t_i}} = \text{sqrt}(\text{normal_error}_{\mathcal{E}_{t_i}})$. Hovering over a point $i > 1$ here shows the residual drift, i.e.

$$\left| \log \frac{\text{RMSE}_{\mathcal{E}_{t_i}}}{\text{RMSE}_{\mathcal{E}_{t_{i-1}}}} \right|$$

- the **Consecutive Tangent-Space Drift**

$$\mathcal{C}(U_{\mathcal{E}_{t_{i-1}}}, U_{\mathcal{E}_{t_i}}) = \sqrt{\sum_{k=1}^d \sin^2 \theta_k}$$

between each retained iteration and its immediate predecessor (principal angles θ_k via `dgesvd` on $U_{\mathcal{E}_{t_{i-1}}}^\top U_{\mathcal{E}_{t_i}}$, restricted to the shared rank $d = d_{t_{i-1}} = d_{t_i}$ – a pair contributes no point if $d_{t_{i-1}} \neq d_{t_i}$, the same “nothing to compare” convention `accept_ensemble`’s own criterion 1 uses; see “SKG `accept_ensemble`” above). At $d = 1$ this reduces to $|\langle u_{t_i}, u_{t_{i-1}} \rangle|$ ’s complement, no `dgesvd` needed. This plot has no point at $t = 1$ (nothing precedes the bootstrap iteration to compare against); it otherwise follows the same x-axis convention as the other two (gap across the evicted range, continuous line across the retained window), so unlike the earlier draft of this section, it aggregates into the “all ensembles” summary below exactly like Spectral Gap and RMSE do – no special-casing needed.

This is a genuinely different quantity from what `accept_ensemble` itself tests: `accept_ensemble`’s criterion 1 compares a candidate against the *whole* reference set $\mathcal{R}$ (bootstrap plus the full trailing window), specifically to catch cumulative drift that a step-to-step-only check would miss (see “SKG `accept_ensemble`” above) – and that whole-reference-set value, as actually computed at each historical growth step, is **not** reconstructable after the fact for any iteration except the most recent one: reconstructing it for iteration i needs the window as it stood right before i was tested, which for $i < T$ has already been evicted by the time growth reaches T . Only for $i = T$ does the currently-stored window (minus its own last column, plus `U_first`) exactly equal the reference set that was actually used to test it – so that one number, the ensemble’s actual accept-tested chordal distance at its final growth step, is shown separately as a caption/subtitle above this plot, not as another line-plot point.

Both numbers (the per-iteration consecutive drift and the final accept-tested value) are computed post-hoc, once per ensemble at report-generation time, directly from the already-exported `ensemble_U_history/ensemble_d_history/ensemble_S_history/ensemble_k_history` arrays (a handful of small `dgesvd` calls, cheap). Neither is stored as part of `ensemble_identification`’s own output – keeping that SKG’s kernels fully general and iteration-unaware, the same reasoning as “First growth step” above: both are visualization-layer derived statistics, not pipeline outputs.

5.2.3.2 “all ensembles” Observable Plots In case the above selection-menu-item has “all ensembles” selected, the above defined plots are generated but show a summary of all ensemble’s plots. Because all ensembles $\mathcal{E}_c$ have at most o observable vectors in their respective Ω_c , we can generate such a plot. Observables which lack some l entries after $t = 1$ and before $T - o + l$ are excluded from contributing information to those l iteration times (x-axis coordinates). At each iteration time t_i calculate the point-wise mean and use it to draw a solid thick black curve. Around it calculate the point-wise standard-deviation and draw it as a shaded uncertainty ribbon. Hovering over points at a given iteration time t_i (x coordinate) opens a small boxplot showing the distribution of values at t_i .

5.3 STC run information pane

Offered as an additional pane in the top horizontal “additional plots” menu-bar is the “SHATTER (STC) run info” pane.

This is not a plot, but an information display. For the current run, a two column table is shown, in which the below information is contained: * All input parameters and their values, including non provided parameters and their default values. Importantly, under each parameter there is a short explanation of the parameter, i.e. what it

controls, its value range (if applicable) and what increasing/decreasing values imply. * The number of input data points, i.e. the size of the ambient vector space so to speak. * Whether parameters were estimated or user-provided. * The number of: * Seeds, * Ensembles, and * Super-Ensembles, and furthermore: * The Min., 1st Qu., Median, Mean, 3rd Qu., and Max. of Ensemble-Sizes. * The Min., 1st Qu., Median, Mean, 3rd Qu., and Max. of Number of Ensembles assigned to each data-point. * The Min., 1st Qu., Median, Mean, 3rd Qu., and Max. of all Ensembles' final observables Spectral-Gap, RMSE, and Tangent-Space Drift.

6 Potential Time Complexity Improvements

observable's SVD (see “SKG **observable**” above) is the leading candidate for where STC spends its time: **grow_ensemble** performs a batch (rank- k , not rank-1) membership expansion per growth iteration, and **observable** recomputes a full economy-mode **dgesdd** decomposition of the ensemble's centered members **from scratch** every time, at every iteration, for every ensemble. Two independent, not-yet-implemented ideas below would each reduce that cost – one requires no algorithmic change at all, the other is a real piece of engineering. Neither has been implemented or profiled; both are recorded here as scoped candidates, not commitments. **Before implementing either, profile a real run first** – both proposals below are asymptotic reasoning, not measurement, and reconciliation's pairwise Overlap Coefficient computation ($O(|\mathcal{E}|^2 \cdot N)$ over all ensemble pairs, see “Ensemble Reconciliation” above) or the k -d tree build could dominate wall time instead, depending on the actual N /ensemble-count/growth-depth regime a given dataset falls into.

6.1 Reduce **normal_error**'s cost via the total-variance identity

normal_error (see “SKG **normal_error**” above) is already described as “free” in the sense that it needs no separate pass over the ensemble's members – it is read directly off the tail of the SVD **observable** already computed. That framing implicitly still assumes the **full** D -length eigenvalue spectrum is available to sum over $j = d_{\mathcal{E}} + 1, \dots, D$. If a future change (see “Batch/incremental SVD via Brand's algorithm” below, or any other truncation) ever stops carrying the full spectrum forward and tracks only the top- $d_{\max}$ tangent directions, **normal_error** would lose its current free ride – unless it is computed a different way:

By the standard Frobenius-norm identity, the sum of *all* eigenvalues of the ensemble's centered covariance equals its total variance:

$$\sum_{j=1}^D \lambda_j^{\mathcal{E}} = \frac{1}{k_{\mathcal{E}} - 1} \sum_{x_i \in \mathcal{E}} \|\mathbf{x}_i - \boldsymbol{\mu}_{\mathcal{E}}\|^2 =: \text{total_variance}_{\mathcal{E}}.$$

So, equivalently,

$$\text{normal_error}_{\mathcal{E}} = \text{total_variance}_{\mathcal{E}} - \sum_{j=1}^{d_{\mathcal{E}}} \lambda_j^{\mathcal{E}}.$$

total_variance is an $O(D)$ running sum of squared deviations from the mean, updatable incrementally as members are added to the ensemble. Precisely – and this is the part worth spelling out, since “Chan/Welford-style” alone does not say which variant, or why it stays $O(D)$ as the ensemble grows:

Maintain two numbers per ensemble across growth, not the full covariance matrix – the running mean $\boldsymbol{\mu}_{\mathcal{E}} \in \mathbb{R}^D$ and a single running **scalar** $M_{\mathcal{E}} := \sum_{x_i \in \mathcal{E}} \|\mathbf{x}_i - \boldsymbol{\mu}_{\mathcal{E}}\|^2$ (so $\text{total_variance}_{\mathcal{E}} = M_{\mathcal{E}} / (k_{\mathcal{E}} - 1)$, read off on demand, not maintained pre-divided). **grow_ensemble** adds a *batch* of k_B new members per iteration, not one point at a time, so the relevant update is the parallel/batch form of this identity (Chan, Golub & LeVeque 1979, “Updating Formulae and a Pairwise Algorithm for Computing Sample Variances”), merging the existing accumulator ($k_A, \boldsymbol{\mu}_A, M_A$) with a fresh one computed directly over just the new batch, ($k_B, \boldsymbol{\mu}_B, M_B$):

$$\delta = \boldsymbol{\mu}_B - \boldsymbol{\mu}_A, \quad \boldsymbol{\mu}_{AB} = \boldsymbol{\mu}_A + \delta \cdot \frac{k_B}{k_A + k_B}, \quad M_{AB} = M_A + M_B + \delta^{\top} \delta \cdot \frac{k_A k_B}{k_A + k_B}.$$

Computing $(\boldsymbol{\mu}_B, M_B)$ fresh over the k_B new members costs $O(D \cdot k_B)$ – unavoidable, since the new coordinates have to be read at least once regardless of algorithm, exactly as **grow_ensemble** already does for every other per-member

quantity. The merge step above is the part that matters: it is $O(D)$ **regardless of how large the existing ensemble $\mathcal{E}$ has already grown** (k_A appears only as a scalar in the weighting, never as a loop bound) – nothing here re-scans the ensemble’s accumulated members, which is exactly what would make this $O(D \cdot k_{\mathcal{E}})$ or worse and erase the saving. Only the top- $d_{\max}$ eigenvalues are then needed to recover **normal_error** – not the full $D - d$ tail – which is exactly what a truncated/incremental tangent basis (below) would leave available.

This change is independent of the SVD-cost idea below and can be adopted on its own regardless of whether **observable** ever moves off a from-scratch **dgesdd** call: it only removes **normal_error**’s dependency on the tail of the spectrum, which today is a non-issue (the tail is already computed anyway) but becomes one the moment the full spectrum stops being computed.

6.2 Batch/incremental SVD via Brand’s algorithm

observable’s **dgesdd** call recomputes the full singular value decomposition of $Y_{\mathcal{E}_{t+1}}$ from scratch at every growth iteration, discarding the previous iteration’s decomposition entirely. Brand’s algorithm (“Fast low-rank modifications of the thin SVD”, Brand 2002) instead *updates* an existing truncated SVD incrementally as new columns (members) are added, without recomputing from scratch:

1. Project the new members onto the current tangent basis U : $p = U^{\top} c$ for each new centered member vector c (**dgemv/dgemm**).
2. Compute the residual outside the current span, $c_{\perp} = c - Up$, and its norm $\rho = \|c_{\perp}\|$ (**dnorm2**).
3. Form and decompose a small bordered matrix built from S , p , and ρ – size $(r + k) \times (r + k)$ for a batch of k new members against a tracked rank r – via an ordinary **dgesvd/dgesdd** call, cheap because r is kept small.
4. Rotate U (and S , and V if kept) through that small SVD’s factors.

Every step is BLAS Level 1/2/3 plus one small dense LAPACK SVD call – no new dependency, and the existing **dgesdd/dgesvd** interface blocks in **observable/accept_ensemble** are already declared **pure** for exactly this reason, so an incremental update drops into the same **pure** kernel discipline with no architectural friction. Because **grow_ensemble** adds a batch of k members per iteration (not one point at a time), the update is the *block* form of Brand’s algorithm, not the simpler rank-1 case.

This is real, scoped engineering, not a drop-in replacement of the **dgesdd** call site – three things are needed beyond the update step itself:

- **A bounded working rank for the spectral-gap search.** **observable** currently discovers $d_{\mathcal{E}}$ by scanning the gap $G(r)$ across *all* $D - 1$ candidate positions (see “SKG **observable**”, step (2)), which needs the full spectrum. An incremental update only tracks a small rank r_{track} (a few more than $d_{\max}$, not the full D), so the gap-search would need to run over that smaller tracked set instead – meaning a rank has to be chosen *before* it is known, which is a real behavior change and a real correctness risk: if the true best gap sits beyond r_{track} (data less structured than expected, or $d_{\max}$ mis-set), the truncated search would silently pick a worse d than today’s full-spectrum scan is guaranteed to find.
- **Mean recentering.** Brand’s original formulation updates a subspace of a *fixed* point set; STC’s $\mu_{\mathcal{E}}$ shifts every time members are added, which needs Brand’s own mean-tracking correction term layered onto the plain projection/update steps above, not just those steps alone.
- **Periodic reorthogonalization.** Over a long growth trajectory, U drifts from exact orthogonality under repeated incremental updates. Since **accept_ensemble**’s accept/reject decisions depend on **chordal_dist**/ G/d being numerically faithful (not merely close), an occasional QR or full-recompute reset is needed to bound that drift – and choosing its cadence is a real design decision, not a default that can be picked once and forgotten.

Combined with the **normal_error** change above, a truncated incremental basis would need to carry forward the top- r_{track} eigenvalues (for the gap search) and the running **total_variance** (for **normal_error**), rather than the full D -length spectrum **observable** currently returns – a detail any implementation of this idea has to account for in its own output shape, not an afterthought bolted on at the end.

6.3 Estimating the local dimension without a full spectral-gap search

The bounded-working-rank caveat above is the one genuine blocker between today’s algorithm and a Brand’s-algorithm-compatible **observable**: r_{track} has to be fixed *before* growth reaches a size where a full-spectrum scan would defeat the point of tracking a small rank at all, but nothing today supplies that number ahead of time. Two distance-based estimators from the intrinsic-dimension-estimation literature could – both give a cheap, per-point or

per-neighborhood estimate of the *local intrinsic dimension* d directly from neighbor distances, with no SVD, no eigendecomposition, and no observable history at all. Both are estimates of d (a number) only, not of the tangent basis U itself – they would seed/bound r_{track} for a subsequent small, Brand’s-algorithm-updated SVD, not replace that SVD.

Levina & Bickel’s maximum-likelihood estimator (Levina, E. and Bickel, P. J., “Maximum Likelihood Estimation of Intrinsic Dimension,” *Advances in Neural Information Processing Systems 17* (NeurIPS 2004), 2005) models the number of neighbors falling within a growing radius around a point as an inhomogeneous Poisson process, and derives the MLE of the local dimension from the log-ratios of nearest-neighbor distances. Given a point x and the distances $T_1(x) \leq \dots \leq T_k(x)$ to its k nearest neighbors (exactly what `kd_knn_query` already returns for seeding/density estimation – no new distance computation needed), the per-point estimate is

$$\hat{m}_k(x) = \left[\frac{1}{k-1} \sum_{j=1}^{k-1} \log \frac{T_k(x)}{T_j(x)} \right]^{-1}.$$

Averaging $\hat{m}_k$ over an ensemble’s own members (or over a range of k , as the original paper recommends to reduce the estimator’s own bias/variance) gives a single, stable local-dimension estimate per ensemble at negligible extra cost over distances STC already has in hand.

TwoNN (Facco, E., d’Errico, M., Rodriguez, A., and Laio, A., “Estimating the intrinsic dimension of datasets by a minimal neighborhood information,” *Scientific Reports* 7, 12140, 2017) goes further and uses only the first two nearest-neighbor distances $r_1(x) \leq r_2(x)$ per point. Under the same local-Poisson-process assumption, the ratio $\mu(x) = r_2(x)/r_1(x)$ follows a Pareto distribution whose shape parameter is exactly the local intrinsic dimension; d is then recovered by a linear fit against the empirical CDF of μ values over a sample of points (a closed-form MLE, or an ordinary least-squares fit on $-\log(1 - F(\mu))$ against $\log \mu$, is enough – no iterative optimization). Needing only 2 neighbors per point makes this the cheaper and more curvature-robust of the two, at the cost of relying on less information per point than Levina-Bickel’s full k -neighborhood.

Neither estimator is exact the way today’s full-spectrum gap search is – both are statistical, with documented bias under non-uniform density, strong curvature, or small sample sizes, the same class of trade Brand’s algorithm itself makes (speed for a guarantee). That argues for treating either as a **starting estimate**, re-checked periodically rather than trusted once: pairing dimension re-estimation with the same cadence as Brand’s algorithm’s own periodic reorthogonalization reset would let one maintenance pass correct both kinds of drift (numerical, in U ; and epistemic, in the assumed d) at once, rather than adding a second, independently-scheduled correction mechanism.